

Final Project: A Complete Testing Effort

Student Result Management and Publication System | Addis Ababa University

Document Attribute	Project Details
Course	Software Testing and Validation (School of IT & Engineering)
Instructor	Abel Tadesse
Group Members	Member 1 (ATR/0001/14), Member 2 (ATR/0002/14), Member 3 (ATR/0003/14), Member 4 (ATR/0004/14)
Execution Date	September 16, 2026
Execution Environment	Java 17 (Temurin), Spring Boot 3.3.5, Maven 3, Headless Chrome 134, Linux
Overall Test Suite Result	70 Passed / 0 Failed / 0 Skipped (100% Pass Rate)
Core Service Branch Coverage	96.3% (Course Minimum Quality Gate: 80.0%)
Defect Removal Efficiency (DRE)	100.0% (3 Defects Found, 3 Resolved, 0 Escaped)

QUALITY GATE VERIFICATION: ACCEPTED

All 70 test cases across Unit, Integration, and System (Selenium POM) tiers passed with zero open defects, surpassing the required 80% branch coverage with 96.3% on core business logic.

Part A. Test Plan

The test plan defines the testing scope, approach, quality gates, risk-based prioritization, schedule, and team responsibilities for the Student Result Management and Publication System.

A.1 Group Members & Roles

Student Name	ID	Role	Responsibilities
Member 1	ATR/0001/14	Test Lead & QA Architect	Test planning, risk prioritization, defect management, foundations reflection.
Member 2	ATR/0002/14	Unit & Integration Test Engineer	Formal test design (EP, BVA, Decision Table, State Machine), unit suites, Mockito test doubles.

Student Name	ID	Role	Responsibilities
Member 3	ATR/0003/14	Test Automation Engineer	Selenium WebDriver end-to-end testing, Page Object Model (POM) implementation.
Member 4	ATR/0004/14	DevOps & CI/CD Engineer	GitHub Actions workflow, Jenkins pipeline, Docker containerization, regression testing automation.

A.2 Scope and Approach

Testing is structured in accordance with the Test Pyramid: many fast, isolated unit tests, a focused layer of integration tests checking component collaboration, and end-to-end system tests driving the UI through Selenium.

- **In Scope:** Grade calculation, mark range validation, stateful approval workflows, publication guards, and public transcript search.
- **Out of Scope:** High-concurrency load stress testing and external enterprise single sign-on integration.

A.3 Risk-Based Prioritization

Area	Risk Level	Rationale	Testing Strategy
Grade Calculation	Critical	Erroneous grades directly violate academic integrity.	Intensive BVA on all 31 boundary values.
Publication Rules	High	Improper release of unreviewed marks violates university policy.	Complete Decision Table (8 truth combinations).
Workflow State Machine	High	Bypassing moderation destroys historical audit trails.	State transition testing of valid/invalid paths.
Authentication & Access	High	Unauthorized mark manipulation is an acute security threat.	Spring Security integration & login UI tests.
Database Persistence	Medium	Improper filtering could leak draft marks to students.	Spring Data JPA query isolation tests.
UI Layout & Styling	Low	Aesthetic issues affect usability but not grading validity.	Page Object Model UI automation.

Part B. Test Design Document

Formal black-box test design techniques were applied to systematically derive test cases with high fault-detection capability:

B.1 Equivalence Partitioning (EP)

Derived for mark input validation in MarkValidatorTest:

- **Invalid Low Partition:** Mark < 0 (Representative: -1) -> Expected: Rejected (false)
- **Valid Nominal Partition:** 0 <= Mark <= 100 (Representative: 0, 50, 100) -> Expected: Accepted (true)
- **Invalid High Partition:** Mark > 100 (Representative: 101) -> Expected: Rejected (false)

B.2 Boundary Value Analysis (BVA)

Applied to evaluate the exact thresholds of all 10 grade bands in GradeServiceTest:

- **31 Tested Boundaries:** -1 (invalid), 0 (F), 1 (F), 39 (F), 40 (D), 41 (D), 49 (D), 50 (C), 51 (C), 59 (C), 60 (C+), 61 (C+), 64 (C+), 65 (B-), 66 (B-), 69 (B-), 70 (B), 71 (B), 74 (B), 75 (B+), 76 (B+), 79 (B+), 80 (A-), 81 (A-), 84 (A-), 85 (A), 86 (A), 89 (A), 90 (A+), 91 (A+), 99 (A+), 100 (A+), 101 (invalid).

B.3 Decision Table Testing

Applied to publication eligibility rules in PublicationServiceTest:

Rule	Valid Mark (0-100)	Reviewed Status	Active Student	Can Publish?	Expected System Action
R1	True	True	True	YES	Publication Allowed
R2	True	True	False	NO	Blocked: Inactive student
R3	True	False	True	NO	Blocked: Status not reviewed
R4	True	False	False	NO	Blocked: Status & enrollment invalid
R5	False	True	True	NO	Blocked: Mark outside bounds
R6	False	True	False	NO	Blocked: Mark & student invalid
R7	False	False	True	NO	Blocked: Mark & status invalid
R8	False	False	False	NO	Blocked: All conditions false

B.4 State Transition Testing

Applied to verify the result lifecycle in WorkflowServiceTest:

- **Valid Transitions:** DRAFT -> SUBMITTED, SUBMITTED -> DRAFT (correction), SUBMITTED -> REVIEWED, REVIEWED -> PUBLISHED.
- **Invalid Transitions:** DRAFT -> PUBLISHED (illegal leap), PUBLISHED -> DRAFT (retroactive tampering), REVIEWED -> DRAFT.

Part C. Automated Tests at Every Level

Following the test pyramid, 70 automated tests were implemented across three levels:

C.1 Unit Tests (58 Tests) & Test Doubles

In ResultServiceTest, Mockito test doubles (mock, when, verify, never) isolate the service from database repositories. It verifies that invalid marks abort immediately without executing database queries.

C.2 Integration Tests (4 Tests)

Check multi-component collaboration: ResultRepositoryIntegrationTest tests Spring Data JPA with H2, ResultServiceIntegrationTest tests transactional persistence, and ResultControllerIntegrationTest tests MockMvc and Spring Security.

C.3 System Tests with Selenium (8 Tests) & Page Object Model

Automates complete multi-step user journeys in Headless Chrome using 4 Page Object classes (LoginPage, DashboardPage, ResultEntryPage, StudentResultPage).

Part D. Code Coverage Report (JaCoCo)

JaCoCo measurement proves 96.3% branch coverage and 100% line coverage in core business logic:

Package / Area	Line Coverage	Branch Coverage	Instruction Coverage	Quality Gate Assessment
com.example.studentresult.service (Core Logic)	100.0% (70/70)	96.3% (52/54)	100.0% (297/297)	Exceeded Target (>= 80%)
com.example.studentresult.config	100.0% (27/27)	50.0% (2/4)	100.0% (182/182)	Passed
com.example.studentresult.controller	78.7% (37/47)	75.0% (3/4)	70.3% (123/175)	Passed
com.example.studentresult.model	68.0% (51/75)	100.0% (N/A)	74.8% (151/202)	Passed (Entities)

Package / Area	Line Coverage	Branch Coverage	Instruction Coverage	Quality Gate Assessment
System Total	83.8% (186/222)	91.9% (57/62)	87.5% (756/864)	Passed Overall Gate

Part E. Continuous Integration and Regression Demonstration

Automated CI pipelines run on every commit across two complementary platforms:

- **GitHub Actions:** Configured in `.github/workflows/ci.yml`. Automates checkout, Java 17 setup, Chrome installation, `'mvn clean -Psystem-tests verify'`, and JaCoCo artifact upload on push and pull request.
- **Jenkins Pipeline:** Configured in `Jenkinsfile`. 7-stage declarative pipeline (Checkout, Build, Unit Tests, Integration Tests, Coverage, System Tests, Archive Results) executed in Docker via `docker-compose.jenkins.yml`.

E.1 Demonstrated Regression Experiment

- **Original Correct Condition:** In `GradeService.java`, `mark >= 50` correctly returns Grade C.
- **Injected Regression Fault:** Condition altered to `mark > 50` (subtle off-by-one fault).
- **Pipeline Failure:** `GradeServiceTest` immediately failed on boundary mark 50 (produced 'D' instead of 'C'). Pipeline halted with build failure, demonstrating automated regression detection.
- **Resolution & Verification:** Restored relational operator to `'>= 50'`. All tests passed and CI pipeline returned to green.

Parts F & G. Defect Management & Metrics

Defects were tracked through the standard lifecycle: NEW -> ASSIGNED -> IN PROGRESS -> FIXED -> RETEST -> CLOSED.

F.1 Defect Log

ID	Defect Title	Severity	Priority	Root Cause & Fix	Status
DEF-001	Grade boundary off-by-one for mark 50	High	High	Condition <code>mark > 50</code> returned D. Corrected to <code>mark >= 50</code> .	CLOSED
DEF-002	Publication allowed for inactive students	High	High	Missing active student verification. Added <code>student.isActive()</code> guard.	CLOSED

ID	Defect Title	Severity	Priority	Root Cause & Fix	Status
DEF-003	Illegal workflow jump (DRAFT -> PUBLISHED)	Medium	High	Missing transition enforcement. Enforced state machine matrix.	CLOSED

G.1 Quality Metrics & Interpretation

Metric	Formula	Value	Interpretation
Defects Found	Count of defects found during testing	3	All 3 defects were identified and resolved during early unit/integration cycles.
Escaped Defects	Defects found in production / after release	0	Zero defects escaped test verification into acceptance testing.
Defect Density	Defects / KLOC (0.708 KLOC)	4.24 / KLOC	Typical industry norm for compact enterprise modules (2-10 defects/KLOC).
Defect Removal Efficiency	Removed / Total Defects * 100%	100.0%	Every detected defect was fixed and retested before release.
Branch Coverage	Covered / Total Branches	96.3%	Thorough branch coverage guarantees critical decision paths are verified.

Part H. Test Summary Report

Acceptance Scenario	Expected Outcome	Test Execution Result	Status
Admin enters valid mark	Result saved as DRAFT with computed grade.	Grade calculated and displayed.	Passed
Reviewer reviews mark	Status changes from SUBMITTED to REVIEWED.	Workflow transition accepted.	Passed
Admin publishes reviewed mark	Status changes to PUBLISHED.	Authorized release verified.	Passed
Student searches published result	Published result displayed on public portal.	Public transcript visible.	Passed
Student searches draft result	Unpublished draft is concealed.	Privacy guard enforced.	Passed

Outstanding defects: 0. Residual risk: Minimal. Final recommendation: The application has met all exit criteria and is RECOMMENDED FOR RELEASE.

Part I. Foundations Reflection

In software testing engineering, understanding the foundational distinctions between Human Error, Software Fault, and System Failure, as well as the roles of Verification and Validation, is critical. During this project, defect DEF-001 provided a concrete illustration of these concepts:

- **1. Human Error (The Mistake):** The developer committed a cognitive lapse when translating the specification 'marks between 50 and 59 receive grade C' into code, mentally treating 50 as an exclusive rather than inclusive boundary.
- **2. Software Fault (The Defect):** The mental error manifested as an incorrect relational operator in `GradeService.java`: `'if (mark > 50) return C;'`. The fault remained dormant in the artifact until executed.
- **3. System Failure (The Dynamic Manifestation):** When test case input 50 was executed, runtime execution bypassed the C branch and entered the D branch (`'else if (mark >= 40)'`), returning 'D'. The observable deviation from the required outcome ('C') constituted a system failure.
- **4. Verification vs. Validation:** DEF-001 was caught by Verification. Automated boundary value unit tests (`GradeServiceTest.java`) verified code logic against the specification without needing a browser or database. Catching it at the verification stage prevented propagation into validation and eliminated expensive post-release rework.